C Programming

Using Pointers: Ch.14

Double pointers | Arrays of pointers | Function pointers | argc/argv

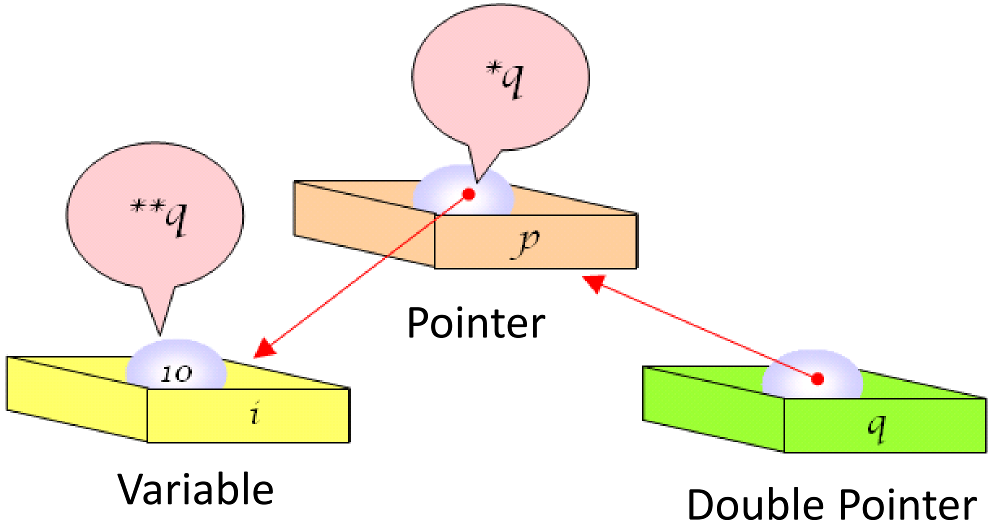
What You Will Learn in This Chapter

- Pointers to pointers, arrays of pointers, and function pointers.
- The relationship between multidimensional arrays and pointers.
- The arguments of the main() function.

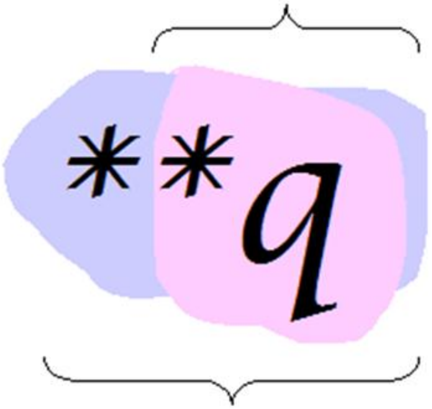
Pointers to pointers: Double Pointer

- Double pointer : a pointer that points to a pointer

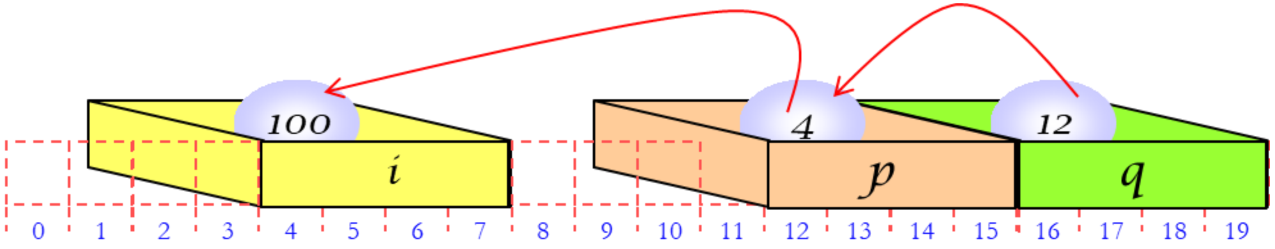
```
int i = 10;      // i is an integer variable
int *p = & i ;  // p is a pointer to i
int **q = &p;  // q is a pointer to p (double pointer)
```



*q : Contents of the location pointed to by q



**q : Contents of the location pointed to by *q



Double Pointer

```
int i = 10;          // i is int type
int *p = & i ;        // p is a pointer to i
int **q = &p;         // q is a pointer to pointer p

int i = 100;
int *p = &i;
int **q = &p;

*p = 200;
printf("i =%d\n" , i );   // 200

**q = 300;
printf("i =%d\n" , i );   // 300 - changed through TWO levels!
```

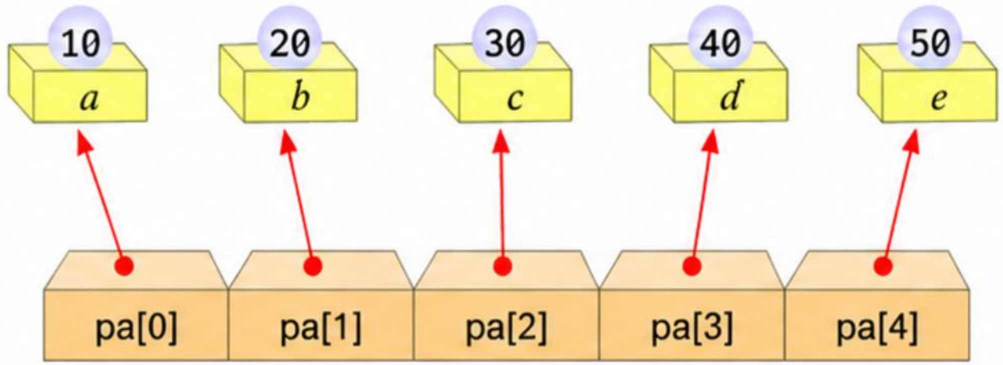
Array of Pointers: Integers & Strings

Arrays can store pointers. Each element of the array is a **pointer**.

1. Array of Integer Pointers

- An array whose elements are pointers to integers.

```
int a = 10, b = 20, c = 30, d = 40, e = 50;
int *pa[5] = { &a, &b, &c, &d, &e };
```



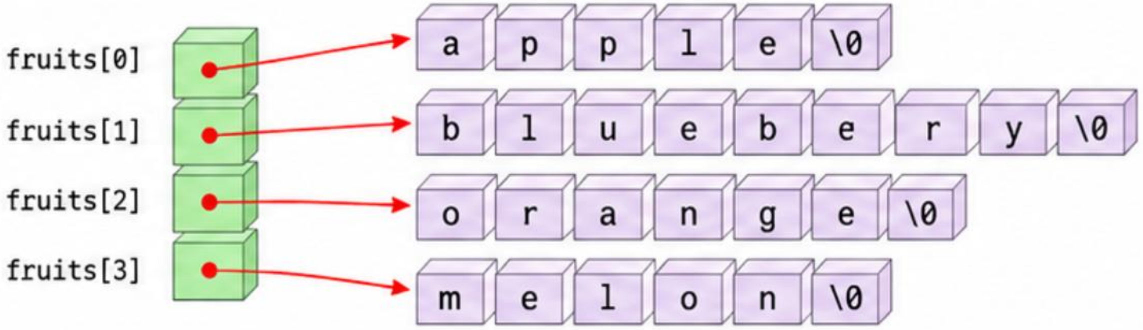
pa (array of 5 integer pointers)

- `pa[i]` is a pointer.
- `*pa[i]` is the integer value.

2. Array of Character Pointers

- An array whose elements are pointers to strings (character arrays).

```
char *fruits[] = {
  "apple",
  "blueberry",
  "orange",
  "melon"
};
```



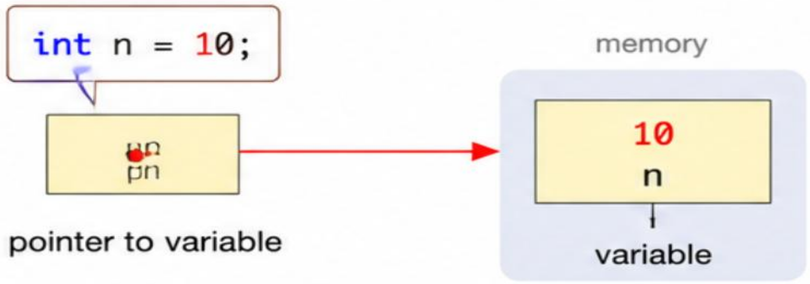
- `fruits[i]` is a pointer to the first character of a string.
- `*fruits[i]` is the first character of that string.

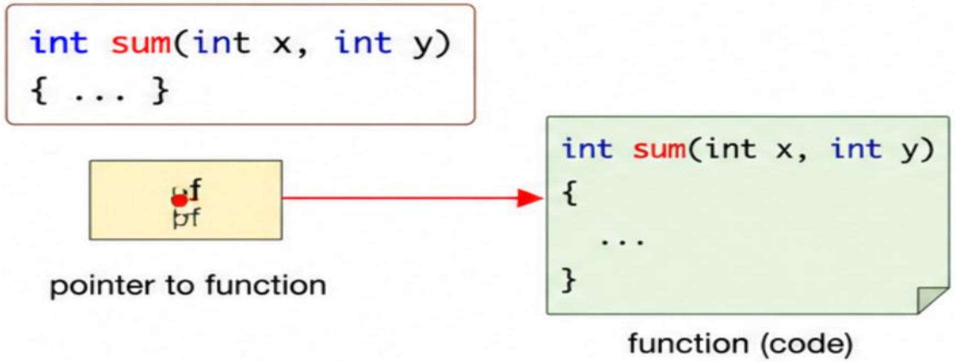
Key Point ✔ Array of pointers → each element is a pointer. | ✔ Integer pointers → point to integer variables. | ✔ Character pointers → point to strings.

Array of Pointers: Functions

- Function Pointer

A pointer that points to **a function.**

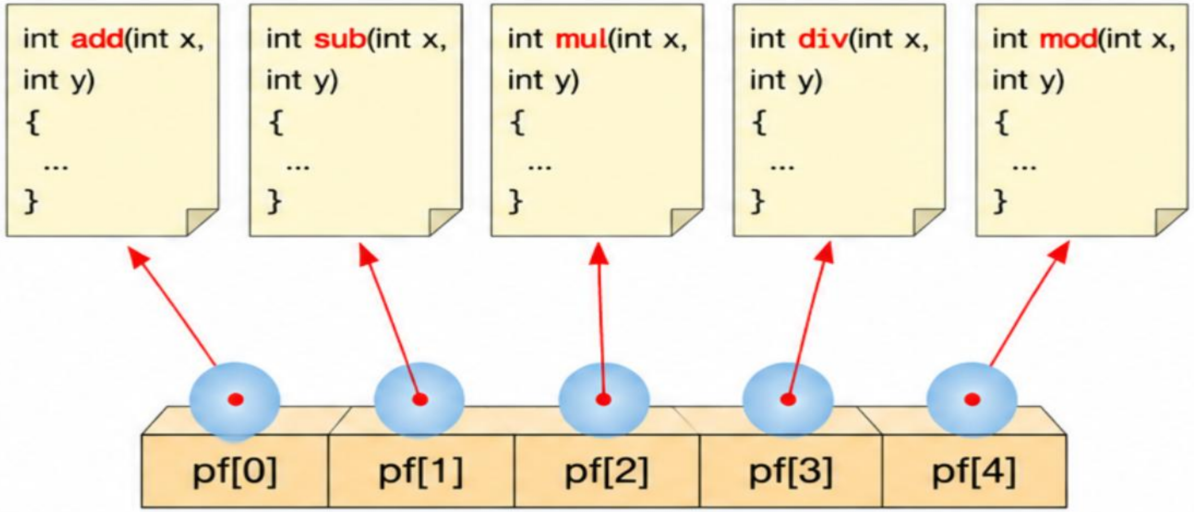




- pn stores the address of a variable.
- pf stores the address of a function.
- pf(x, y) calls the function pointed by pf.

3. Array of Function Pointers

An array whose elements are **pointers to functions.**




```
int (*pf[5])(int, int) = { add, sub, mul, div, mod };
```

- pf[i] is a pointer to a function.
- pf[i](x, y) calls the function pointed by pf[i].
- Example: pf[0](a, b) calls add(a, b).

String Array

```c
#include < stdio.h >

int main( void )
{
    int i , n;
    char *fruits[ ] = {
        "apple" , "blueberry" , "orange" , "melon"
    };

    n = sizeof (fruits)/ sizeof (fruits[0]); // count elements

    for ( i = 0; i < n; i ++)
        printf ( "%s \n" , fruits[ i ]);
    return 0;
}
```

The main() Function's Arguments

main() Function Format

1. main() Function

The main() function format so far

```c
int main(void)
{
    ...
}
```

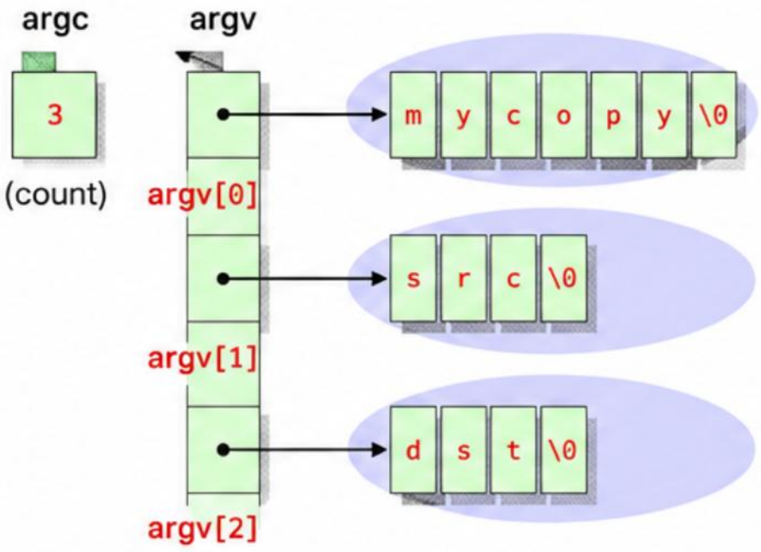
The main() function that receives input from outside

```c
int main(int argc, char *argv[])
{
    ...
}
```

- argc : number of command line arguments
- argv : array of character pointers
- argv[0] : program name
- argv[1] ~ argv[argc-1] : arguments

2. How Input is Passed

```
C:\> mycopy src dst
```



- argv[0] : "mycopy"
- argv[1] : "src"
- argv[2] : "dst"

3. Example (main_arg.c)

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    int i = 0;

    for (i = 0; i < argc; i++)
        printf("%dth string on command line = %s\n",
               i, argv[i]);

    return 0;
}
```

```
C:\>program> mainarg src dst
0th string on command line = mainarg
1st string on command line = src
2nd string on command line = dst
C:\>program>
```

- argc = 3
- argv[0] = "mainarg"
- argv[1] = "src"
- argv[2] = "dst"

main.c: Reading Command-Line Arguments

```c
#include < stdio.h >

int main( int argc , char * argv [])
{
    int i = 0;
    for ( i = 0; i < argc ; i ++)
        printf ( "command line argument %d = %s\n" , i , argv [ i ]);
    return 0;
}
// When run as:  C> main arg1 arg2 arg3
// argc = 4,  argv[0] = "main", argv[1] = "arg1", ...
```

Q & A

Lab & Quiz: right after this - see you in the practice session!